

Da caçada às cegas ao diagnóstico científico: debugging assistido com Claude Code

Resumo da pesquisa feita após a prática "O Especialista em Código Assistido por IA e o Gerente de Engenharia Cético" — como o Claude Code entende blocos inteiros de código legado, rastreia dependências ocultas e propõe correções sem quebrar o sistema.

Fernando Melo · 21 de setembro de 2026

01 · O PROBLEMA

Código legado, bugs difíceis e um autocomplete que não basta

O cenário parte de um gerente de engenharia cético: o time passa muito tempo com código legado e bugs difíceis, e o autocomplete tradicional já não resolve. A pergunta central é qual capacidade fundamental permite ao Claude Code "entender" e analisar blocos de código inteiros — algo que modelos menores não fazem — e como isso reduz drasticamente o tempo para encontrar e corrigir bugs complexos.

02 · A CAPACIDADE FUNDAMENTAL

Janela de contexto gigante + execução agêntica

A capacidade que permite ao Claude Code ir além do autocomplete é a combinação de uma janela de contexto extensa — de 200.000 a 500.000 tokens — com uma arquitetura de execução agêntica. Ela funciona como uma memória de trabalho de curto prazo massiva (equivalente a centenas de páginas de texto), permitindo carregar simultaneamente o histórico da sessão, múltiplos arquivos do repositório, logs do console e resultados de testes.

Modelos menores

Analísam apenas a linha atual ou o arquivo aberto — sem visão do restante do repositório nem do histórico da sessão.

Claude Code

Carrega histórico da sessão, vários arquivos do repositório, logs de console e resultados de testes ao mesmo tempo.

LSP: buscas cirúrgicas em vez de força bruta

Contexto gigante sem precisão gera alucinação e retrabalho. Por isso o Claude Code não faz uma leitura bruta de todo o repositório jogando gigabytes de texto na janela de contexto — isso faria a atenção do modelo se perder, um fenômeno chamado *context rot* (degradação de contexto). Em vez disso, ele inicia em segundo plano um servidor LSP (Language Server Protocol) correspondente à stack do projeto (Pyright para Python, JDT.LS para Java, gopls para Go), que transforma o código de texto puro em um mapa estruturado de árvore sintática, permitindo buscas cirúrgicas por definições, tipos e referências exatas com o mínimo de tokens.

Exemplo prático — dependência oculta: um sistema de e-commerce falha de forma intermitente ao processar pagamentos pesados. O log mostra apenas uma linha:

```
Error: Connection timeout at OrderProcessor.js:142
```

O desenvolvedor olha a linha 142 e vê só uma chamada HTTP padrão. O problema real está escondido em um middleware de telemetria legado que intercepta a requisição três camadas abaixo e trava o pool de conexões sob estresse.

- 01 Varredura via LSP: o Claude lê o arquivo na linha indicada e usa o LSP para mapear o objeto de requisição (ex.: `httpClient`) até onde ele foi instanciado.
- 05 Proposta cirúrgica: exibe o diff da correção sugerida (ex.: tornar a gravação de métricas totalmente assíncrona com um fallback seguro) e aguarda autorização para aplicar e criar uma nova branch no Git.

04 • FLUXO DE TRABALHO

4 etapas: sintoma, investigação, validação, commit

Para transformar a correção de bugs de uma "caçada às cegas" em um diagnóstico científico, o desenvolvedor deixa de ser um digitador de código e passa a atuar como investigador-chefe, validando as hipóteses levantadas pela IA.

- 1 **Entrada do sintoma** — o desenvolvedor não precisa isolar o arquivo do bug: inicia o Claude Code na raiz do repositório enviando o sintoma bruto (log de erro, ID de issue ou comportamento reportado pelo QA).
- 2 **Investigação da causa raiz** — em vez de sugerir o próximo caractere, o Claude Code inicia um raciocínio lógico em cadeia, varrendo o projeto e perguntando a si mesmo no terminal ("De onde vem esse objeto?", "O que altera esse comportamento antes do envio?").
- 3 **Validação do patch** — roda o comando de teste do projeto para confirmar a hipótese antes de propor qualquer mudança definitiva.
- 4 **Revisão e commit** — encerra o fluxo apresentando um `git diff` detalhado e pergunta se o desenvolvedor aprova as mudanças.

Trecho ilustrativo simplificado

```
claude "/bug O endpoint /orders/checkout está falhando com Erro 500 no ambiente
de staging quando o carrinho possui cupons de desconto antigos. Veja os logs recentes."

# Executado automaticamente pelo Claude Code sob o capô:
npm test -- src/legacy/coupon_validator.test.js

# diff proposto, aguardando aprovação (y):
+ if (!coupon.expiry_date) return false; // cupom sem data de validade é tratado como inválido
```

Como não quebrar o que já está frágil

Em sistemas legados e frágeis, consertar um bug frequentemente quebra três outras funcionalidades. Para mitigar esse risco, o Claude Code usa três camadas de proteção:

CAMADA	COMO FUNCIONA
Arquivo de governança (CLAUDE.md)	Regras locais na raiz do projeto: padrões de código exigidos (ex.: "mantenha o padrão ES5 deste repositório"), comandos de validação obrigatórios e zonas proibidas de alteração.
Ciclo fechado de testes	Propõe a alteração, aplica o patch temporariamente, executa a suíte de testes; se falhar (regressão), lê o log de erro, desfaz a alteração e tenta uma nova abordagem.
Mudanças cirúrgicas e isoladas	Em vez de refatorar o arquivo inteiro, aplica o menor impacto possível e só avança após apresentar o <code>git diff</code> e obter aprovação humana.

Fim do "chutômetro" em código legado

Preserva a integridade da aplicação: a IA só avança se a suíte de testes locais der sinal verde. **Documentação instantânea da falha:** ao final, basta pedir "Explique o que causou esse bug para eu colocar no histórico da Issue" para receber um resumo técnico da causa raiz. **Elimina o "chutômetro":** o time para de alterar linhas aleatórias torcendo para o erro sumir — o processo se torna rigoroso e documentado.